

1. Definir la arquitectura backend basada en capas

La arquitectura está basada en tres diferentes capas HTTP Request, Controller, Service, Repository, DataBase que es el estándar y el patrón de diseño de una arquitectura API REST, ya que se necesita un backend robusto y separado para alimentar la aplicación móvil.

2. Explicar el rol de cada capa

- a. HTTP Request: Es lo que haría el usuario que utiliza la aplicación móvil hacia el backend.
- b. Controller: Esta capa es la que orquesta todo del backend, procesa cada solicitud HTTP que manda la aplicación móvil, su función es meramente recibir y validar el HTTP y redireccionarlo junto a las demás capas. Esta capa no tiene nada de lógica, validaciones, ni cálculos, es ordenar la solicitud. Y esta capa responde a cada request HTTP con un código 200, 201 ect.
- c. Service: Esta capa se encarga de todo la parte lógica de la aplicación, tiene reglas propias que el negocio necesita como que se pueda hacer una reservación, y reglas lógicas que requiere la aplicación. Esta capa también coordina con múltiples repositorios para el acceso a varias tablas y validación de datos con los DTOs para poder hacer la presentación al usuario.
- d. Repository: Esta capa se encarga de la abstracción de los datos en el motor de la base de datos, ejecuta las query con las diversas operaciones que se necesiten, además se puede utilizar mapper para facilitar las queries. Es la única capa que debe interactuar con el motor de base de datos.

3. Describir el flujo de una petición desde el cliente hasta la base de datos

1. La app Android envía una petición HTTP POST con cuerpo JSON al endpoint de la API.
2. La Controller recibe la petición.
 - a. Se procesa el JSON en un objeto (DTO).
 - b. Se validan los campos obligatorios
 - c. Si la validación falla se retorna 400 Bad Request inmediatamente.
3. El Controlador invoca al Service para la operación correspondiente
4. El Service ejecuta la lógica de negocio cálculos y validaciones que se requiera
5. Si todo sale bien el Service llama al repository para hacer la operación de la query
6. El Repository ejecuta la Query en la base de datos

7. La base de datos confirma la operación realizada y da esa información de vuelta al Repository
8. El Servicio convierte la respuesta en una entidad un DTOs
9. El Controlador retorna la respuesta con el cuerpo JSON.
10. La app Android recibe la respuesta y actualiza la interfaz de usuario.

4. Justificar las decisiones tomadas

Arquitectura en capas: Con una clara separación de las responsabilidades de cada capa puede probarse o escalar de forma independiente sin afectar a las demás. Lo que permite una mejor trazabilidad de toda la aplicación y además un mejor mantenimiento, dejando en claro la parte de cada capa hace una única cosa, teniendo todo ordenado y si surgen problemas es más fácil de rastrear.

API REST: Al ser una aplicación en API REST, sin estado, que está basada en HTTP, es fácil de poder consumir desde Android en este caso. Lo que permite una mayor facilidad y escalabilidad en un futuro.

DTOs: Al usar un repositorio con DTOs se evita la exposición de campos internos de las entidades, permiten previsualizar el contenido de la API sin romper los clientes existentes y una mayor protección de los datos, y solo se usa lo que se solicita.